

Trabalho Prático.

1 Introdução

A utilização de código intermediário permite desacoplar a geração da sequência básica de instruções das particularidades e do formato de uma arquitetura de destino. Além disso, a estratégia de geração de código prioriza uma abordagem genérica, funcional e didática, que pode ser posteriormente aprimorada por meio de diferentes técnicas de otimização antes da etapa de geração do código em linguagem de montagem (assembly) de uma arquitetura alvo. Assim, para fixar o entendimento neste trabalho, é solicitada a implementação (descrita na próxima seção) de um back-end para geração de código intermediário, na forma de um código de três endereços.

2 O Que Deve Ser Implementado

Nesta fase da disciplina, como trabalho prático, deverá ser modificado o projeto implementado na unidade anterior de modo a:

- Introduzir uma implementação da estrutura de 3AC (Código de Três Endereços) – Deve ser desenvolvido um módulo responsável por representar instruções de código de três endereços. Cada instrução será composta por um código de operação e três referências para entradas da tabela de símbolos, possibilitando o encadeamento dessas instruções em listas. Cada 3AC descreve a execução de uma operação com zero, um ou dois operandos armazenados em memória, produzindo igualmente um resultado armazenado em memória. Os endereços de memória utilizados pelos operandos e pelo resultado são representados por símbolos registrados na tabela de símbolos. Além da estrutura de representação, o módulo deve disponibilizar funções auxiliares para criação, impressão e concatenação de listas de instruções.
- Criação de símbolos temporários e labels — O novo módulo deve disponibilizar duas rotinas auxiliares para a inserção de novos símbolos na tabela de símbolos (hash): uma destinada à criação de variáveis temporárias e outra à criação de labels. As variáveis temporárias serão utilizadas para armazenar resultados intermediários produzidos durante a geração de código, enquanto as labels servirão para identificar pontos de desvio e controle do fluxo de execução do programa.
- Geração de código — Deve ser implementada uma rotina responsável por percorrer recursivamente a Árvore Sintática Abstrata (AST), produzindo, para cada nodo visitado, um

trecho de código intermediário representado por uma lista de instruções 3AC. O processamento deve iniciar pelos nós filhos, armazenando os trechos de código gerados para cada subárvore. Em seguida, a rotina analisa o nodo corrente e gera as instruções correspondentes à sua semântica. Em geral, esse processo envolve criar uma ou mais instruções 3AC, a concatenação dos trechos de código provenientes das subárvores com as novas instruções geradas e, quando necessário, a criação de símbolos temporários e labels auxiliares. Como resultado, a rotina retorna um trecho de código intermediário completo correspondente ao nodo processado.

3 Entrega e Critérios de Avaliação.

O projeto deverá ser apresentado até o dia 10/07. Durante a apresentação, cada equipe deverá demonstrar o funcionamento da implementação, explicando as principais decisões de projeto adotadas e exemplificando a geração de código intermediário a partir de programas de entrada.

A avaliação considerará principalmente os seguintes aspectos:

1. Qualidade do código intermediário gerado, verificando a correção, a organização e a fidelidade das instruções 3AC produzidas em relação à semântica do programa-fonte.
2. Apresentação do trabalho, considerando a clareza da exposição, a demonstração prática da implementação, a capacidade de justificar as decisões de projeto e o domínio do funcionamento da solução desenvolvida.